Phi4Chatbot Core System Documentation

1. System Architecture Overview

The `Phi4Chatbot` backend application is a production-grade Web API built on the **ASP.NET Core Minimal APIs** framework. It handles conversational text streams, provides real-time user feedback tracking, routes incident ticketing, and enforces rigorous safety guardrails [cite: 1].

Storage Architecture

- **LiteDB (Embedded NoSQL):** Handles ephemeral data modeling, including conversational text caches, instant user feedback indices, and security guardrail log feeds [cite: 1].
- **SQL Server LocalDB (Relational DB):** Implements formal persistence for transaction-isolated tracking of engineering support logs and tickets [cite: 1].
- **IMemoryCache (In-Memory Layer):** Manages rapid absolute-expiration lookups for security correlation IDs [cite: 1].

2. Security Framework & Guardrails

The backend includes multiple protective validation subsystems deployed sequentially to filter runtime input/output vectors [cite: 1]:

Guardrail Subsystem	Validation Strategy	Action taken on Violation
Input Length Constraint	Blocks payloads exceeding 500 characters [cite: 1].	Logs as `InputValidation`; Returns HTTP 400 Bad Request [cite: 1].
Control Character Filter	Detects hidden, embedded ASCII or Unicode control bytes [cite: 1].	Logs as `InputValidation`; Returns HTTP 400 Bad Request [cite: 1].
Prompt Injection Layer	Applies deterministic Regex heuristics to block bypass keywords (e.g., `"ignore all previous instructions"`, `"developer mode"`) [cite: 1].	Logs as `PromptInjection`; Flushes dynamic block warning [cite: 1].
Harmful Content Shield	Configurable switch checking requests against patterns for malware, cyberattacks, weapons, or illegal synthesis [cite: 1].	Logs as `HarmfulContent`; Returns generic safe refusal [cite: 1].
Model Refusal Monitor	Evaluates completions via standard compliance subroutines to catch `"i cannot assist"` patterns [cite: 1].	Logs as `ModelRefusal` for continuous alignment audit [cite: 1].
Output Leakage Guard	Scans structural model outputs for core systemic leaks (e.g., system prompts, operational keys) [cite: 1].	Prevents system caching to mitigate risk vectors [cite: 1].

3. Core Web API Enclaves

A. Chat Completion & RAG Streaming (`POST /api/chat`)

Processes inbound requests, leverages local textual context lookup from the `Documents/` directory using word-frequency score matching, injects custom defensive system prompts, and pipes chunked token completions to the front-end via **Server-Sent Events (SSE)** [cite: 1].

B. Feedback Tracking (`POST /api/chat/feedback`)

Ingests individual transaction user signals (likes `1` or dislikes `-1`) against active `AnswerId` cache indices to build datasets for Reinforcement Learning alignment loops [cite: 1].

C. Ticket Engineering Logging (`POST /api/chat/ticket`)

Generates unique ticket identifiers (e.g., `TKT-[Timestamp]-[Random_String]`) for negative user experiences, saving them directly to SQL Server database tables [cite: 1].